

Platform Rebuild Proposal — Executive Summary

Purpose: Convince management to approve an incremental platform rebuild that reduces version sprawl, speeds delivery, and makes client customization and integrations sustainable.
Audience: Executive leadership, product, engineering, professional services, support.
Date: June 2026

1. Core recommendation

Rebuild the platform **incrementally** using a **strangler-fig** approach: a single canonical core with **versioned extension and integration packs** — not a big-bang rewrite and not continued forking per client.

Ask:

- Charter a **Platform 2.0** program with dedicated capacity (not 20% time).
- Approve a **3–6 month foundation + pilot** phase with measurable exit criteria.
- Adopt policy: **no new git submodules, no new core forks, no new SaveChanges workflow handlers.**

2. The problem (in business terms)

We do not have a feature problem. We have a **variant problem**.

Symptom	Business impact
Simple changes take weeks	Missed commitments, unhappy customers
Regressions in unrelated areas	Support load, emergency releases
Integrations behave differently per client	High PS cost, slow onboarding
Cloud + on-prem + customized builds	Too many versions to test and support
“Which version is running?”	Long incidents, risky upgrades
New hires slow to contribute	Delivery does not scale with headcount

One line: *We are paying a growing tax on every feature because customization and integrations were implemented as compile-time forks, not runtime configuration.*

3. Root technical causes

1. **Git submodules and client-specific code branches** multiply release combinations.
2. **Class inheritance for services** spreads logic across opaque override trees.
3. **Per-client data model forks** make migrations, APIs, and reporting unsustainable.
4. **EF SaveChanges change handlers** implement hidden workflows (legacy of Access DB + stored procedures).
5. **Hangfire jobs** duplicate and defer logic that should be explicit orchestration.
6. **Weak domain boundaries** — WBS, planning, hours, imports, and integrations are coupled.

This is normal debt from years of shipping under pressure. It is not a people problem.

4. Strategic goal: one platform, many configurations

Dimension	Target
Deployment	Same product artifact for cloud SaaS and on-prem (profile-driven config).
Integration	Connector packs (SAP, PLM, HR, file, etc.) — not core forks.
Customization	Configuration, policy hooks, and extension actors — not submodules.

Success criterion: A new client integration or customization ships as a **versioned pack** enabled per tenant, not as a separate product build.

5. Why the current customization model fails

Approach	Long-term result
Git submodules per client/integration	Version matrix explosion, merge hell
Subclassing services	Fragile base classes, hard-to-find overrides
Custom EF models per client	Forked migrations, APIs, reports, upgrades
if (tenant == X) in core	Accidental complexity, untestable branches

Replacement principle: *Professional services may customize behavior, not the core release artifact.*

6. Target architecture: platform core + packs



Core owns: canonical domain, invariants, APIs, persistence boundaries, external ID registry.

Packs own: client- or system-specific mapping, validation, enrichment, connectors, and policy steps.

7. Akka.NET: why it fits (orchestration, not fashion)

Akka.NET is proposed because our hardest problems are **workflows**, not CRUD: import/export pipelines, integration retries, long-running recalculations, and tenant-specific processing steps.

Actors vs. inheritance / submodules

Legacy	Akka approach
AcmeImportService : ImportService	ImportPipeline → [MapActor, AcmeRulesActor, PersistActor]
Client submodule	Tenant profile registers actor pack
Hidden handler chain	Explicit message pipeline with correlation ID

Actors vs. SaveChanges handlers + Hangfire

Legacy pattern	Problem	Akka replacement
EF change handlers on save	Implicit order, nested saves, hard to test	Command → orchestrator actor → persist actor

Hangfire jobs with business logic	Two workflow systems, duplicated rules	Supervised connector/workflow actors
Scheduled sync	Job-centric observability	Scheduler triggers actor message

Rule: SaveChanges is for **persistence**, not for **business process orchestration**.

```

Command (ImportProject, BookHours, RecalculatePlan)
  → Orchestrator actor
    → Persist actor (sole EF boundary for that workflow)
    → Integration / rollup / notification actors
    → Domain events (ImportPersisted, PlanRecalculated)

```

What Akka is NOT for: simple CRUD, DbContext in every actor, or replacing a disciplined domain model. Use actors at **boundaries**.

8. Data model strategy (avoid per-client schema forks)

1. **One canonical schema** for all customers on **SQL Server** (production and on-prem standard).
2. **Governed extension data** for rare attributes (registry + validation).
3. **External ID mapping** — source systems map to internal entities without alternate keys.
4. **Read models / projections** for client-specific reporting.

Customize **behavior and mappings**, not the physics of the database except through extension points.

Datastore: SQL Server only — no SQLite or per-client schema forks. Shared-database tenants use TenantId + indexes; enterprise tenants may use dedicated SQL Server instances (same schema, different connection).

9. Cloud and on-prem

Same binary; differences are **configuration and enabled packs** (identity, messaging, storage, updates, offline file import/export).

10. Proof of concept

An internal POC demonstrates: external IDs, actor-orchestrated persistence, template-first ordering, planning with FS/SS/FF/SF + lag, domain value types, and automated tests.

Message: We are asking to scale an approach that already produced working vertical slices.

11. Phased migration

- **Phase 0:** Inventory handlers, jobs, submodules; define parity and packs.
- **Phase 1:** Contracts, actor registry, tenant profiles; freeze new handlers/submodules.
- **Phase 2:** Pilot domain in production (import/WBS); retire one legacy path.
- **Phase 3:** Expand domains; publish compatibility matrix.

Hangfire may remain temporarily as a **scheduler trigger** only during migration.

12. Metrics

Track: active forks, lead time, change failure rate, time to add integration, “unknown version” tickets, on-prem upgrade rate, % work delivered as packs vs forks.

13. Options

Option	Verdict
A. Status quo	Cheapest now; cost accelerates
B. Incremental rebuild (recommended)	Balanced risk; continuous delivery
C. Big-bang rewrite	Highest revenue and execution risk

14. Executive summary

Our product’s complexity is driven less by feature count than by years of client-specific customizations and integrations, delivered through git submodules, service inheritance, customized data models, EF change handlers, and Hangfire jobs — patterns inherited from an earlier Access/stored-procedure architecture. Combined with cloud and on-prem delivery, this created too many de facto product versions to test, upgrade, and support.

*The proposed rebuild shifts to a **platform-and-packs architecture**: a stable canonical core; integrations as connector packs; customizations as tenant-configured actor pipelines; deployment profiles for cloud versus on-prem from the **same build**. Long-running work moves from implicit SaveChanges side effects and background jobs into **explicit, supervised Akka.NET workflows** with clear message contracts.*

This restores predictable delivery, reduces support matrix size, protects professional services margin, and gives customers a credible upgrade path without forking the product for each engagement.

16. Performance and large projects (40–50,000 activities)

Customers range from small projects (~40 activities) to large programmes (~50,000 activities). Responsiveness is a **product architecture** requirement, not only a hardware decision.

Design principle

Interactive paths stay small; heavy work runs asynchronously on bounded slices.

Path	Target	Pattern
Open project / browse WBS	< 500 ms	Paginate, lazy-load tree, summaries
Edit one activity	< 100 ms	Targeted read/write
Full plan recalculation (large project)	Seconds, background	Planning actor + progress events
Gantt view	Smooth scroll	Virtualized rows + visible time window
Bulk import	Minutes OK	Actor pipeline + batched upsert + progress

Database (SQL Server)

- **Never load full project tree by default** — lazy expansion, cursor pagination, search endpoints.
- **Indexes:** (TenantId, ProjectId), (ComponentId), (ActivityId), assignment and relation FKs.
- **Component-level rollups** — cached budgeted hours, % complete, date ranges.
- **Dedicated SQL Server** for large enterprise tenants (isolation tier).

API tiers

Endpoint style	Purpose
/projects/{id}/summary	Counts, dates, rollups only

/components?parentId=	One tree level at a time
/activities?projectId=&cursor=	Paginated flat list / search
/projects/{id}/plan?from=&to=	Windowed Gantt data

Return **projections**, not full entity graphs, to the UI.

Planning at scale

- **Store schedule snapshots** — UI reads persisted plan; does not recalculate on every page load.
- **Async recalculation** via `PlanningCoordinatorActor` for projects above threshold (e.g. 500 activities).
- **Incremental scheduling** — on single edit, recalculate affected subgraph only; full recompute on import or bulk relation change.
- **Never block HTTP** on full 50k-activity forward pass.

UI

- Virtual scrolling for Gantt and activity lists.
- Time-windowed Gantt (load visible months, fetch more on pan).
- Component-level aggregation with expand-for-detail.
- Debounced plan triggers (one background job per burst of edits).

Imports

- Batched upsert (e.g. 500 rows per transaction) through import actor pipeline.
- Progress events (`ImportProgressUpdated`) for UI feedback.
- Idempotent external IDs for safe retry.

Multi-tenant isolation

- **Performance tiers** in tenant profile: standard vs large.
- Rate limits and fair queuing for heavy jobs (import, replan).
- Large tenants: dedicated DB + worker capacity.

SLOs (proposal targets)

- Up to **500 activities**: interactive edits < 1 second.
- Up to **50,000 activities**: browse and edit remain interactive; full replan in background with progress within minutes.
- Load-test gates at **40, 500, 5,000, and 50,000** activities before major client migrations.

17. Decision requested

1. Approve **Platform 2.0** program charter and dedicated team.
2. Fund **discovery + foundation + one pilot domain**.
3. Enforce **no new submodules / handler workflows**.
4. Standardize on **SQL Server** for cloud and on-prem data stores.
5. Adopt **performance SLOs** for large projects (Section 16).
6. Name **executive sponsor** (engineering + product + commercial).
7. Select **pilot customer** or internal flagship project for first cutover.

Document generated from engineering strategy discussions. Internal use.